Matrix-Free Preconditioned Conjugate Gradients for an RKHS-Constrained CP Mode Update with Missing Data

Abstract

We consider the mode-k subproblem arising in alternating optimization for a rank-r CP decomposition of a d-way tensor with missing entries, where the kth factor lies in a reproducing kernel Hilbert space (RKHS) and is parameterized as $A_k = KW$ with a positive (semi)definite kernel matrix $K \in \mathbb{R}^{n\times n}$. The resulting normal-equation system has dimension $nr \times nr$ and involves a selection operator for the $q \ll N$ observed tensor entries. We explain how to solve this system efficiently by a matrix-free preconditioned conjugate gradient (PCG) method. We derive an operator form for the system matrix, show how to compute matrix–vector products in $O(qr + n^2 r)$ time without forming any N-scale objects, and propose a Kronecker-structured preconditioner that is inexpensive to apply and typically yields rapid convergence.

1 Problem statement and notation

Let $\mathcal{T} \in \mathbb{R}^{n_1\times\cdots\times n_d}$ be a partially observed ($q \ll N$) tensor, where $N = \prod_{i=1}^{d} n_i$. Fix a mode $k \in \{1,\dots,d\}$ and write $n \equiv n_k$ and $M \equiv \prod_{i\neq k} n_i$ so that the mode-k unfolding $T \in \mathbb{R}^{n\times M}$ satisfies $\mathrm{vec}(T) \in \mathbb{R}^N$ with $N = nM$.

Let $S \in \mathbb{R}^{N\times q}$ be a *selection matrix* whose columns are distinct standard basis vectors in $\mathbb{R}^N$ so that $S^T \mathrm{vec}(T) \in \mathbb{R}^q$ extracts the observed entries of the unfolding. Equivalently, $P \coloneqq SS^T \in \mathbb{R}^{N\times N}$ is a diagonal projector with ones on observed coordinates and zeros elsewhere.

Let $Z \in \mathbb{R}^{M\times r}$ be the Khatri–Rao product of all CP factors except mode k:

$$Z \;=\; A_d \odot \cdots \odot A_{k+1} \odot A_{k-1} \odot \cdots \odot A_1,$$

and let $B = TZ \in \mathbb{R}^{n\times r}$ denote the MTTKRP.

Assume an RKHS parameterization for mode k:

$$A_k = KW, \qquad K \in \mathbb{R}^{n\times n} \text{ (psd kernel matrix)}, \quad W \in \mathbb{R}^{n\times r} \text{ unknown.}$$

The linear system to be solved is

$$\Big[(Z \otimes K)^T SS^T (Z \otimes K) + \lambda(I_r \otimes K)\Big] \operatorname{vec}(W) = (I_r \otimes K) \operatorname{vec}(B), \qquad (1)$$

where $\lambda > 0$.

A direct solve on the explicit $nr \times nr$ matrix costs $O((nr)^3) = O(n^3 r^3)$ flops and requires $O((nr)^2)$ storage, which is typically prohibitive. Moreover, explicitly forming the matrix would require operating with S and $Z \otimes K$ at the N-scale.

We assume

$$n, r < q \ll N = nM,$$

and we seek a method whose dominant costs scale with q and n rather than N.

2 Structure of the system matrix

Define

$$A \;\coloneqq\; (Z\otimes K)^T P(Z\otimes K)+\lambda(I_r\otimes K), \qquad b \;\coloneqq\; (I_r\otimes K)\operatorname{vec}(B) = \operatorname{vec}(KB).$$

Then (1) is $Ax = b$ with $x = \operatorname{vec}(W)$.

2.1 SPD property and suitability of (P)CG

Lemma 1 (Positive semidefiniteness)**.** *The matrix $(Z \otimes K)^T P(Z \otimes K)$ is symmetric positive semidefinite.*

Proof. Since $P = SS^T$ is symmetric positive semidefinite (a diagonal projector),

$$(Z \otimes K)^T P(Z \otimes K) = \big(S^T(Z \otimes K)\big)^T \big(S^T(Z \otimes K)\big)$$

is a Gram matrix, hence symmetric positive semidefinite. □

Lemma 2 (Positive definiteness under mild conditions)**.** *If $K \succ 0$ (strictly positive definite) and $\lambda > 0$, then $A \succ 0$ and the conjugate gradient method is applicable.*

Proof. Because $K \succ 0$, the Kronecker product $I_r \otimes K \succ 0$, hence $\lambda(I_r \otimes K) \succ 0$ for $\lambda > 0$. Adding a positive semidefinite matrix preserves positive definiteness:

$$A \;=\; (Z \otimes K)^T P (Z \otimes K) + \lambda(I_r \otimes K) \;\succ\; 0.$$

□

Remark 1 (If K is only psd)**.** *If $K \succeq 0$ is singular, then $\lambda(I_r \otimes K)$ does* not *regularize directions in* $\ker(K)$*, and A may be singular. In practice one either (i) adds a small nugget $K \leftarrow K + \varepsilon I$ (common in kernel methods), or (ii) restricts the unknown to the range of K (working with a factorization of K). In what follows we assume $K \succ 0$ for simplicity, which is the standard setting for CG/PCG.*

3 Matrix-free matrix–vector products

The efficiency of an iterative solver hinges on computing $y = Ax$ without forming A.

3.1 Kronecker identities

We will repeatedly use the standard identity: for conforming matrices A, B, X,

$$(B \otimes A)\,\mathrm{vec}(X) = \mathrm{vec}(AXB^T). \tag{2}$$

In particular, if $X \in \mathbb{R}^{n\times r}$ then

$$(Z \otimes K)\,\mathrm{vec}(X) = \mathrm{vec}(KXZ^T) \in \mathbb{R}^{nM} = \mathbb{R}^N.$$

Also, for $Y \in \mathbb{R}^{n\times M}$,

$$(Z^T \otimes K)\,\mathrm{vec}(Y) = \mathrm{vec}(KYZ) \in \mathbb{R}^{nr}.$$

3.2 Masking interpretation of $P = SS^T$

Let $\Omega \subseteq [n] \times [M]$ denote the observed index set in the unfolding, with $|\Omega| = q$. Define the binary mask matrix $\mathcal{M} \in \{0,1\}^{n\times M}$ by $\mathcal{M}_{i,m} = 1$ iff $(i,m) \in \Omega$. Then, by construction of S,

$$P\;\mathrm{vec}(Y) \;=\; \mathrm{vec}(\mathcal{M} \odot Y), \tag{3}$$

where $\odot$ on the right denotes elementwise (Hadamard) multiplication.

3.3 Operator form for A

Proposition 1 (Operator representation)**.** *For any* $X \in \mathbb{R}^{n\times r}$,

$$A\ \mathrm{vec}(X) = \mathrm{vec}\Big(K\big(\mathcal{M} \odot (KXZ^T)\big)Z\ +\ \lambda KX\Big). \tag{4}$$

Proof. Using (2) and (3),

$$(Z\otimes K)^T P(Z\otimes K)\,\mathrm{vec}(X) = (Z^T\otimes K)\ \mathrm{vec}\big(\mathcal{M}\odot(KXZ^T)\big) = \mathrm{vec}\Big(K(\mathcal{M}\odot(KXZ^T))Z\Big).$$

Moreover $(I_r \otimes K)\,\mathrm{vec}(X) = \mathrm{vec}(KX)$, hence the λ term is $\mathrm{vec}(\lambda KX)$. Summing gives (4). □

Equation (4) already shows that we never need to form any $N \times N$ objects: we only need to (i) multiply by K on the left (an $n \times n$ operation), and (ii) apply the mask $\mathcal{M}$, which is sparse with q nonzeros.

3.4 Avoiding explicit formation of Z

Although $Z \in \mathbb{R}^{M\times r}$ is enormous when M is huge, *we do not form it.* In sparse CP computations, one stores the observed tensor entries as a list of multi-indices

$$\alpha = (i_1, \ldots, i_d) \in [n_1] \times \cdots \times [n_d], \qquad \alpha \in \Omega_{\mathrm{ten}}, \quad |\Omega_{\mathrm{ten}}| = q,$$

together with values $\mathcal{T}_\alpha$.

For each observed tensor entry α, the corresponding row of the Khatri–Rao matrix Z (in the unfolding column indexed by the other-mode tuple) is the Hadamard product

$$z_\alpha\ :=\ A_1(i_1,:) \odot \cdots \odot A_{k-1}(i_{k-1},:) \odot A_{k+1}(i_{k+1},:) \odot \cdots \odot A_d(i_d,:)\ \in\ \mathbb{R}^r, \tag{5}$$

where $A_\ell(i_\ell,:)$ denotes the $1 \times r$ row of factor A_ℓ. Thus any access to Z at observed locations can be done in $O((d-1)r)$ time per observation by forming z_α.

3.5 A q-sparse formula for the masked products

Let $U := KX \in \mathbb{R}^{n\times r}$. For an observed unfolding entry corresponding to tensor index α with mode-k coordinate $i = i_k$, the scalar

$$s_\alpha\ :=\ (KXZ^T)_{i,m(\alpha)}\ =\ \langle U(i,:), z_\alpha\rangle,$$

Algorithm 1 Matrix-free product $y = A\ \mathrm{vec}(X)$

Require: $X \in \mathbb{R}^{n\times r}$, kernel matrix $K \in \mathbb{R}^{n\times n}$, observed tensor index list Ω_{ten} of length q, fixed factors $\{A_\ell\}_{\ell\neq k}$, regularization $\lambda > 0$.

Ensure: $Y \in \mathbb{R}^{n\times r}$ such that $\mathrm{vec}(Y) = A\ \mathrm{vec}(X)$.

```
U ← KX                                  (cost O(n^2 r) if K dense; O(nnz(K) r) if sparse)
V ← 0_{n×r}
for each observed tensor entry α = (i_1, ..., i_d) ∈ Ω_ten do
    i ← i_k
    Form z_α ∈ R^r via (5)              (cost O((d − 1) r))
    s ← ⟨U(i, :), z_α⟩                  (cost O(r))
    V(i, :) ← V(i, :) + s z_α^T         (cost O(r))
end for
Y ← KV + λU                             (cost O(n^2 r) plus a saxpy)
return Y
```

where $m(\alpha)$ is the unfolding column index determined by the other-mode tuple. Then the matrix

$$V \;:=\; (\mathcal{M} \odot (KXZ^T))\, Z \in \mathbb{R}^{n\times r}$$

can be accumulated without forming either $\mathcal{M} \odot (KXZ^T)$ or Z:

$$V(i,:) \;=\; \sum_{\alpha\in\Omega_{\text{ten}}:\, i_k=i} s_\alpha\, z_\alpha. \tag{6}$$

Finally, $A\,\mathrm{vec}(X) = \mathrm{vec}(KV + \lambda U)$.

3.6 Matrix–vector product algorithm and complexity

Algorithm 1 implements $y = A\ \mathrm{vec}(X)$ in a matrix-free way.

Per-matvec cost. Assuming dense K, the dominant costs are:

- Two kernel multiplications (KX and KV): $2 \cdot O(n^2 r)$.
- A single pass over the q observations. Each observation costs $O((d-1)r)$ to form z_α, plus $O(r)$ for the dot product and $O(r)$ for the rank-one update, i.e. $O(dr)$ per observation:

$$O(qdr) \quad \text{(often written as } O(qr) \text{ when } d \text{ is treated as constant).}$$

Thus each matrix–vector product costs

$$T_{\mathrm{mv}} = O(n^2 r + qdr). \tag{7}$$

This avoids any $O(N)$ operation because the only loop is over Ω_{ten} (size q), not over all nM unfolding entries.

Memory. One stores K ($O(n^2)$ if dense), factor matrices $\{A_\ell\}$ ($O(\sum_{\ell \neq k} n_\ell r)$), and the sparse tensor index/value list ($O(q)$ plus values). No N-length vectors are formed.

4 Preconditioned conjugate gradients

4.1 PCG method

Because $A \succ 0$ under Lemma 2, the conjugate gradient (CG) method can be applied. Since the conditioning of A may be poor (kernel matrices can be ill-conditioned; missingness can exacerbate this), one typically uses *preconditioned* CG.

Let $M \succ 0$ be a preconditioner approximating A and inexpensive to invert (apply). PCG applies CG to the equivalent system

$$M^{-1}Ax = M^{-1}b$$

in a way that preserves symmetry via the M-inner product. A standard PCG iteration is shown in Algorithm 2.

4.2 Why PCG is efficient here

A direct method scales as $O((nr)^3)$ and requires forming/storing an $nr \times nr$ matrix. In contrast, PCG requires only:

1. A matrix-free application of A (Algorithm 1), costing $T_{\mathrm{mv}} = O(n^2 r + qdr)$ per iteration.
2. Application of the preconditioner $z = M^{-1}r$, which we choose to be similarly inexpensive.

If PCG converges in m iterations (often $m \ll nr$ when preconditioned), the total cost is roughly

$$T_{\mathrm{PCG}} \approx m\,(T_{\mathrm{mv}} + T_{\mathrm{prec}}) + T_{\mathrm{setup}},$$

which can be orders of magnitude smaller than $O(n^3 r^3)$.

Algorithm 2 Preconditioned conjugate gradients for $Ax = b$

Require: SPD operator A (via Algorithm 1), RHS b, SPD preconditioner M with solver $z = M^{-1}r$, initial guess x_0, tolerance ε.

```
r_0 ← b − A x_0
z_0 ← M^{-1} r_0
p_0 ← z_0
ρ_0 ← r_0^T z_0
for j = 0, 1, 2, ... until convergence do
    q_j ← A p_j
    α_j ← ρ_j / (p_j^T q_j)
    x_{j+1} ← x_j + α_j p_j
    r_{j+1} ← r_j − α_j q_j
    if ‖r_{j+1}‖_2 ≤ ε‖b‖_2 then
        break
    end if
    z_{j+1} ← M^{-1} r_{j+1}
    ρ_{j+1} ← r_{j+1}^T z_{j+1}
    β_j ← ρ_{j+1} / ρ_j
    p_{j+1} ← z_{j+1} + β_j p_j
end for
return x_{j+1}
```

5 Choice of preconditioner

The missing-data projector P destroys the exact Kronecker structure of the full-data normal equations. A common and effective strategy is to use a preconditioner that *restores* this structure by replacing the mask with its mean behavior. We describe a widely used Kronecker preconditioner and show how to apply it efficiently.

5.1 A full-observation (Kronecker) approximation

If all entries were observed, then $P = I_N$ and the system matrix would be

$$A_{\text{full}} = (Z \otimes K)^T (Z \otimes K) + \lambda(I_r \otimes K) = (Z^T Z) \otimes (K^T K) + \lambda(I_r \otimes K). \quad (8)$$

Since K is symmetric, $K^T K = K^2$. Define the $r \times r$ Gram matrix

$$G \;\coloneqq\; Z^T Z \succeq 0.$$

Then

$$A_{\text{full}} = G \otimes K^2 + \lambda I_r \otimes K. \quad (9)$$

For missing-at-random patterns, P is often heuristically close (in an averaged sense) to a scalar multiple of I_N, so (9) is a reasonable approximation to A up to scaling. This motivates the preconditioner

$$M \;:=\; G \otimes K^2 + \lambda I_r \otimes K, \tag{10}$$

possibly multiplied by an overall scalar (which does not affect the preconditioned condition number).

5.2 Computing $G = Z^T Z$ without forming Z

Even though Z is enormous, its Gram matrix is small ($r \times r$) and can be computed from the other factors.

Lemma 3 (Gram of a Khatri–Rao product)**.** *Let* $Z = A_d \odot \cdots \odot A_{k+1} \odot A_{k-1} \odot \cdots \odot A_1$. *Then*

$$Z^T Z \;=\; \mathop{*}_{\ell \neq k} \left(A_\ell^T A_\ell\right),$$

where $*$ *denotes the Hadamard (elementwise) product.*

Proof. For two factors, $(A \odot B)^T (A \odot B) = (A^T A) * (B^T B)$ is standard and follows by expanding inner products row-wise; induction yields the multi-factor result. □

Thus G is obtained by forming the $r \times r$ Gram matrices $A_\ell^T A_\ell$ for $\ell \neq k$ and multiplying them elementwise. This costs

$$O\left(\sum_{\ell \neq k} n_\ell r^2\right)$$

and requires only $O(r^2)$ memory for G.

5.3 Applying M^{-1} efficiently

To use PCG we need to apply $z = M^{-1} r$ for vectors $r \in \mathbb{R}^{nr}$.

Using (2), for any $X \in \mathbb{R}^{n \times r}$ we have

$$(G \otimes K^2)\,\mathrm{vec}(X) = \mathrm{vec}(K^2 X G), \qquad (\lambda I_r \otimes K)\,\mathrm{vec}(X) = \mathrm{vec}(\lambda K X),$$

so $M\,\mathrm{vec}(X) = \mathrm{vec}(K^2 X G + \lambda K X)$. Thus applying M^{-1} amounts to solving a (generalized) Sylvester-type equation

$$K^2 X G + \lambda K X = R, \qquad \text{where } R \in \mathbb{R}^{n \times r} \text{ is the reshaped residual.} \tag{11}$$

A convenient way to solve (11) exploits simultaneous diagonalization in tensor-product bases.

Eigen-based application. Assume $K \succ 0$ and compute once per outer ALS step:

$$K = V\Lambda V^T, \qquad \Lambda = \mathrm{diag}(\kappa_1, \ldots, \kappa_n),\ \kappa_j > 0,$$

$$G = U\Gamma U^T, \qquad \Gamma = \mathrm{diag}(\gamma_1, \ldots, \gamma_r),\ \gamma_i \geq 0.$$

Then M is diagonalized by $U \otimes V$:

$$M = (U \otimes V)\,\mathrm{diag}\big(\gamma_i\kappa_j^2 + \lambda\kappa_j\big)_{j,i}\,(U \otimes V)^T,$$

so

$$M^{-1} = (U \otimes V)\,\mathrm{diag}\Big(\frac{1}{\gamma_i\kappa_j^2 + \lambda\kappa_j}\Big)_{j,i}\,(U \otimes V)^T. \tag{12}$$

Applying $z = M^{-1}r$ can be done without any Kronecker matrices by reshaping $r = \mathrm{vec}(R)$ and using (2) in matrix form:

$$(U^T \otimes V^T)\,\mathrm{vec}(R) = \mathrm{vec}(V^T R U).$$

Thus:

1. Reshape r to $R \in \mathbb{R}^{n\times r}$.
2. Compute $\widetilde{R} \leftarrow V^T R U$.
3. Divide elementwise: $\widetilde{X}_{j,i} \leftarrow \widetilde{R}_{j,i}/(\gamma_i\kappa_j^2 + \lambda\kappa_j)$.
4. Compute $X \leftarrow V\widetilde{X}U^T$ and return $z = \mathrm{vec}(X)$.

Cost of applying M^{-1}. With dense V, the dominant costs are the two basis transforms $V^T R$ and $V\widetilde{X}$, each $O(n^2 r)$, plus the small right transforms by U ($O(nr^2)$). Hence

$$T_{\mathrm{prec}} = O(n^2 r + nr^2). \tag{13}$$

The *setup* costs to compute the eigendecompositions are $O(n^3)$ for K and $O(r^3)$ for G; these are typically amortized over many PCG iterations within the mode-k update.

Remark 2 (Cheaper variants)**.** *If n is large and $O(n^3)$ setup is too expensive, one may use an approximate factorization of K (e.g. incomplete Cholesky / Nyström) and apply an approximate M^{-1}. Another simple alternative is a block-diagonal preconditioner that retains only* $\mathrm{diag}(G)$*, yielding r decoupled $n \times n$ systems involving K; this can be effective when component cross-correlations are mild.*

6 Right-hand side and avoiding Z in $B = TZ$

While the focus is the linear solve, note that the right-hand side $b = \mathrm{vec}(KB)$ requires $B = TZ$. With missing entries, one *never* forms T or Z explicitly; instead, B is computed by a sparse MTTKRP pass:

$$B(i_k, :) += \mathcal{T}_\alpha \, z_\alpha^T, \qquad \forall \alpha = (i_1, \ldots, i_d) \in \Omega_{\text{ten}},$$

where z_α is defined by (5). This costs $O(qdr)$ and produces $B \in \mathbb{R}^{n \times r}$; then KB costs $O(n^2 r)$.

7 Overall complexity and comparison

Let m be the number of PCG iterations needed to reach a desired relative residual tolerance. The dominant costs are:

Per PCG iteration.

$$T_{\text{iter}} = T_{\text{mv}} + T_{\text{prec}} = O(n^2 r + qdr) \; + \; O(n^2 r + nr^2) = O(qdr + n^2 r),$$

since typically $nr^2 \leq n^2 r$ when $n \geq r$.

Setup per outer ALS step.

$$T_{\text{setup}} = O(n^3)\,(\text{EVD/Cholesky of } K) + O(r^3)\,(\text{EVD of } G) + O\left(\sum_{\ell \neq k} n_\ell r^2\right) \; (\text{Grams for } G).$$

Total solve cost.

$$T_{\text{PCG}} = T_{\text{setup}} \; + \; m\, O(qdr + n^2 r). \tag{14}$$

Direct method cost. A dense direct solve of (1) is $O((nr)^3) = O(n^3 r^3)$, and forming the matrix explicitly would require manipulating objects with $N = nM$ rows (or $q \times nr$ sketched matrices), which is infeasible when N is enormous.

Why this avoids $O(N)$ computation. All operations in (14) involve:

- Dense $n \times n$ kernel multiplications ($O(n^2 r)$),
- Small $r \times r$ Gram operations ($O(r^3)$ and $O(r^2)$),
- Passes over the observed entries only ($O(qdr)$).

No step iterates over all N tensor entries, forms $Z \in \mathbb{R}^{M \times r}$, or allocates N-length dense vectors.

8 Concluding remarks

The key to efficiency is to treat (1) as an SPD system and apply PCG in a matrix-free manner. The missingness operator $P = SS^T$ is implemented implicitly by looping over the q observed entries. The Kronecker identities (2) reduce the action of $(Z \otimes K)$ and its transpose to kernel multiplications and right-multiplication by Z restricted to observed indices, which can be performed using Khatri–Rao row products (5) without forming Z.

A Kronecker-structured preconditioner (10) based on the full-data normal equations yields a fast approximate inverse that can be applied in $O(n^2 r)$ time per iteration after an $O(n^3)$ kernel factorization. Under the regime $n, r < q \ll N$, this reduces the mode-k solve from $O(n^3 r^3)$ to approximately $O(m(qr + n^2 r))$ (up to constants depending on d), with m typically modest in practice due to preconditioning.